Engineer Take-Home Exercise

Context System for Intelligent Routing

Overview

At Arcpoint, we're building the intelligent control plane for autonomous optimization of the next generation of compute. Our mission is to route every inference request to the optimal model, at the right time, on the right infrastructure.

The challenge is immense: truly intelligent routing is impossible without a rich, timely, real-time understanding of the world.

This exercise focuses on the **Context Layer**, the core system responsible for ingesting signals, building an ontology of the system, and responding to events necessary for high-stakes, instantaneous routing decisions. This is where pragmatic systems design meets product and business thinking.

The Challenge of Real-Time Intelligence

The core problem isn't just collecting data. It's transforming a torrent of signals into actionable intelligence in milliseconds. The system must operate with complete knowledge in the face of continuous change, enabling the routing engine to answer questions like:

Dynamic State: "What is the true, current availability and performance profile of every model and backend at this precise moment?"

Personalized Routing: "Given this user's SLA, quota, and historical preference, plus current global system load and pricing, which backend offers the best trade-off for this request?"

Causality & Foresight: "How can we connect a quality drop hours ago to a routing decision made today, and what capacity risks are we taking in the next 60 minutes?"

Your task is to design and prototype the Context Layer. This is open-ended. We want you to define the necessary data, architect the flow, choose the right persistence layers for varied query patterns, and establish the APIs that let the routing engine access personalized, real-time context on every request.

Time Expectation

4-6 hours is the target. You're welcome to invest more time if you find the problem interesting, but we don't expect polish. We want to see how you think, build, and make decisions. Clear reasoning, identifying trade-offs, and making good choices are what matter.

On Using AI

We strongly encourage you to use AI tools throughout this exercise. At Arcpoint, AI engineering is how we work, and we value people who use these tools to move faster and produce better work.

- Use Claude, Codex, OpenCode, or whatever tools you find helpful. We want to see YOUR workflow.
- AI-generated code is welcome. What matters is that you understand it and can explain your choices.

- We're evaluating your judgment, system design thinking, and ability to ship working software.

Problem Statement

Our routing engine needs to answer questions like:

- *"What's the current state of our model fleet?"* Which models are available, degraded, or down?
- *"Which backend should I use for this request?"* Given current load, costs, and user requirements.
- *"Why did quality drop for reasoning tasks last Tuesday?"* Connecting outcomes to decisions.
- *"What will traffic look like in the next hour?"* Informing capacity decisions.

Your task is to design and prototype a context system that supports these queries. This includes thinking about what data you need, where it comes from, how you store and access it, and how an agent or application can consume it.

Scenario

Assume the following environment:

- **10 models** available for routing (mix of proprietary and open-source, different capabilities/costs)
- **3 compute backends** (different cloud providers/regions: AWS, k8s cluster, neocloud)
- **~10K requests/day** with variable traffic patterns and spikes
- **Multiple user tiers** with different SLAs and cost tolerances
- **Quality scores** available async (hours later) for a subset of requests

49.36.69.15 2026-04-05

Data Universe

Part of this exercise is deciding what data matters. Below are categories of information that could be relevant. You won't use all of it, and you might identify gaps.

Infrastructure State

- **Model health:** Availability status, error rates, current latency percentiles, rate limits remaining
- **Backend metrics:** Current load, queue depth, spot instance availability, regional latency
- **Cost signals:** Current pricing (may be dynamic), budget burn rates, cost anomalies
- **Incidents:** Active incidents, recent postmortems, maintenance windows

Request & Traffic Data

- **Request logs:** Every request with metadata, routing decision, and outcome
- **Traffic patterns:** Historical volume by hour/day, task type distribution, user tier mix
- **Quality feedback:** Async evaluation scores, user thumbs up/down, error reports
- **User context:** Tier, quota usage, historical preferences, custom routing rules

Operational Context

- **Model metadata:** Capabilities, pricing, context windows, known limitations
- **Routing policy:** Current routing rules, A/B test configurations, feature flags
- **Business rules:** Cost ceilings, quality floors, compliance requirements

- **External signals:** Provider status pages, announced deprecations, new model releases

Design Considerations

- **Freshness vs. cost:** Some data needs to be real-time, some can be batch. What's the right refresh cadence for each?
- **Data availability:** Not all data is equally accessible. Consider API rate limits, async pipelines, and missing fields.
- **Query patterns:** Point lookups vs. aggregations vs. time-series analysis. Different access patterns need different storage.
- **Agent consumption:** How does an LLM-based agent interact with this context? Tool calls? RAG? Structured prompts?
- **Operator experience:** What does a human operator need to see? Dashboards, alerts, drill-downs?

Deliverables

You have flexibility in how deep you go. Choose one of the following paths, or a thoughtful mix:

49.36.69.15_20260305

Option 1: Total System Design

A detailed, end-to-end architecture covering Parts 1 and 2, focusing on data flow, storage choices, and API contracts.

Option 2: Agent-Centric Context Engine

Focus on Part 3: a working prototype of the Context API and an LLM-based agent that queries and interprets the context for routing decisions.

Option 3: ML-Augmented Routing

A system design and/or prototype that integrates ML models (for traffic forecasting, quality prediction, or dynamic cost-benefit analysis) as first-class citizens within the Context Layer.

What matters is clear thinking and the ability to make progress on an ambiguous problem. The work should be coherent and easy to communicate.

Exercise Parts

Part 1: Data & Context Design

1. What data do you actually need? What can you ignore?
2. Where does each piece of data come from? What are the access patterns?
3. How do you handle data that's stale, missing, or unreliable?
4. What's your storage strategy? (Think about query patterns, not just "use Postgres")

Part 2: Detailed System Design

1. How does the context system integrate with the routing engine?
2. How would an agent query this system? What tools/APIs does it need?
3. How do you handle real-time vs. analytical queries?
4. What does the operator experience look like?

Expected artifacts:

- Architecture diagrams showing components and data flow
- API contracts / schema definitions

- Technology choices with justification
- Execution plan: What would you build first? What are the risks?

Part 3: Prototype

1. Build a working subset of your design
2. Could be: a context API, an agent that answers questions about system state, a data pipeline, or a simple dashboard
3. Demonstrate that your approach works with synthetic or mock data

Evaluation Criteria

We're evaluating how you approach a complex, ambiguous problem. An excellent submission shows:

- **Architectural Pragmatism:** A well-justified system design that clearly differentiates between real-time, near-real-time, and analytical data flows.
- **The Dimension of Time:** Deep consideration for change over time. How does your system track model drift, changing cost curves, and evolving traffic patterns? How does it handle stale, missing, or unreliable data?
- **Personalized Speed:** A concrete approach to making context lookup fast and tailored to each request, without adding unacceptable latency.
- **Product Sense:** A clear understanding of what matters to the routing engine, operators, and the business (cost, quality, availability).
- **Strong Opinions:** Clear opinions backed by solid reasoning. We value decisive justification over generic "it depends" answers.
- **Evidence of Execution:** Tangible proof of work. A small but working prototype or a concrete, realistic design document with clear technology choices.

Submission Guidelines

- Submit as a GitHub repository shared with **@ramin**
- Use a clean, linear commit history (not a single commit)
- Include a README with an overview of your approach and how to run any code
- Treat the presentation of your repo as production quality: well organized, no stray files, easy to navigate
- Design docs and written portions can be markdown files in the repo
- Commit history should reflect your process. We like to see how work evolves.

Questions? Reach out if something is genuinely unclear, but we intentionally leave room for interpretation. How you handle ambiguity is part of what we're evaluating.

Good luck! We're excited to see how you approach this.

Prepared for 2026-02-05